

TP N°7 : Recherche dans une Base de Données.

Partie1: Les fonctions d'agrégations (Count, Avg, Sum, Max, Min, Distinct) – La Jointure.

Objectif: Le but de ce TP est de découvrir les différentes opérations SQL permettant d'améliorer la tâche de la recherche dans une base de données Oracle, dans ce contexte, nous allons manipuler les fonctions d'agrégations pour regrouper/traiter les valeurs de plusieurs lignes afin de former une seule valeur récapitulative. Dans la suite de ce TP, nous allons découvrir l'utilisation des jointures pour extraire les informations de deux ou plusieurs tables dans une seule requête.

Définition :

Une fonction d'agrégation SQL est utilisée pour effectuer des calculs sur plusieurs lignes d'une seule colonne d'une table et renvoyer une seule valeur.

▼ La fonction : COUNT()

La fonction COUNT() est utilisée pour compter le nombre de lignes d'une table spécifiée qui satisfont une condition.

Syntaxe :

```
SELECT COUNT(*|column_name)
FROM table_name
WHERE condition;
```

On utilise :

- ▶ COUNT(*): Pour compter le nombre totale de lignes d'une table.
- ▶ COUNT(column_name): Pour compter le nombre de lignes qui ne sont pas nulles dans une colonne particulière d'une table.

On considère la relation suivante: Employe (CodeE, NomE, PrenomE, Fonction, Salaire).

Exemple 01: COUNT(*)

– Quel est le nombre total d'employés?

```
SELECT COUNT(*) FROM Employe ;
```

Exemple 02: COUNT([DISTINCT] column)

– Quel est le nombre total d'employés ayant un poste?

```
SELECT COUNT(fonction) FROM Employe;
```

Ça retourne le nombre de lignes dont la colonne fonction n'est pas nulle.

– Pour afficher le nombre des valeurs distinctes de la colonne fonction :

► **SELECT COUNT(DISTINCT fonction) FROM Employe;**

PS. Vous pouvez utiliser l’alias **AS** pour attribuer temporairement un nouveau nom à votre résultat de SELECT et donc simplifier la lecture du résultat renvoyé.

Exemple :

```
SELECT COUNT(*) AS Nombre_Employe FROM Employe;
```

Nombre_Employe est le nom que on va donner à notre résultat grâce à l’alias **AS**.

▼ Les fonctions : MAX() & MIN()

Renvoient respectivement la valeur la plus haute et la plus basse dans une colonne particulière.

Syntaxe :

```
SELECT MAX|MIN(column_name)
FROM Table_name
WHERE Condition;
```

La fonction MIN() : détermine la plus petite valeur de toutes les valeurs sélectionnées d'une colonne. Cette fonction récupère l’enregistrement le plus petit.

Exemple 01:

– Quel est le salaire minimum pour tous les employés?

► **SELECT MIN (salaire) FROM Employe ;**

La fonction MAX() : détermine la plus grande valeur de toutes les valeurs sélectionnées d'une colonne. Cette fonction récupère l’enregistrement le plus grand.

Exemple 02:

– Quel est le salaire maximum pour les employés au poste «A1» ?

► **SELECT MAX (salaire) FROM Employe WHERE fonction= 'A1';**

▼ La fonction : SUM()

La fonction SUM() est utilisée pour calculer la somme de toutes les valeurs sélectionnées dans une colonne. Cela ne fonctionne que sur les champs numériques.

Syntaxe :

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

Exemple :

– Quel est la somme totale des salaires ?

► **SELECT SUM (salaire) FROM Employe;**

▼ La fonction : AVG() – moyenne

La fonction `AVG()` permet de calculer la valeur moyenne de toutes les valeurs sélectionnées dans une colonne du type numérique.

Syntaxe :

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

Exemple :

Quel est le salaire moyen des employés occupant le poste 'A1' ?

► `SELECT AVG(salaire) AS Salaire_Moy FROM Employe WHERE fonction= 'A1';`

La Jointure:

▼ i.e. : Joindre deux ou plusieurs tables avec un attribut commun

Une jointure est une opération qui consiste à extraire les informations/données de deux ou plusieurs tables dans une seule requête.

Exemple 01 : Joindre deux tables

► Afficher les noms de tous les produits commandés.

```
SELECT Distinct P.nom_P
FROM Produit P, Commande C
WHERE P.code_P = C.code_P ;
```

Ici, on a besoin de manipuler la table *Produit* afin de récupérer les noms & la table *Commande* pour sélectionner les codes des produits existants dans la table commande, puis appliquer la jointure entre les deux tables (mettre un égale entre les attributs communs des deux table).

Exemple 02 : Joindre 3 tables

► Afficher les noms des clients ayant commandé le produit « Prd1 ».

```
SELECT DISTINCT C.nom_C1
FROM client C, commande Cm, produit P
WHERE C.num_C1= Cm.num_C1 AND P.code_P=Cm.code_P AND P.nom='Prd1';
```

Pour répondre à la requête, il est nécessaire de manipuler les trois tables (client, commande et produit)

Auto jointure : (Joindre une table avec elle-même)

Exemple 03:

► Quel sont les produits ayant le même prix que le produit numéro 4.

```
SELECT P1.nom_P
FROM produit P1, produit P2
WHERE P1.prix = P2.prix AND P2.code P = 4 AND P1.code P <> 4 ;
```

Ici, il est nécessaire de manipuler la table *Produit* deux fois avec des noms différents (P1 et P2) et faire la jointure entre deux avec l'attribut commun (prix).

TP N°8 : Recherche dans une Base de Données.

Partie2: Sous-requête, les clauses: Order by, Group by-Having, Exists, Union et Intersect.

Objectif: Le but du TP est d'apprendre à manipuler les différentes opérations SQL permettant d'améliorer la tâche de recherche dans une base de données Oracle, dans ce contexte, nous allons découvrir les sous requêtes, le tri de résultat de SELECT (Order by) et le partitionnement des tables (Group by et Having).

La sous requête – Subquery



Une sous-requête, aussi appelée « requête imbriquée », consiste à exécuter une requête à l'intérieur d'une autre requête. Elle est souvent exprimée au niveau d'une clause WHERE pour remplacer une ou plusieurs constantes.

Syntaxe :

```
SELECT column_name(s)
FROM table_name(s)
WHERE [Attr|Instruction] Opérateur (SELECT column_name
                                   FROM table_name(s)
                                   WHERE condition );
```

Les données renvoyées par la sous-requête sont utilisées par l'instruction externe de la requête de niveau supérieur, appelée requête principale.

Une sous-requête doit renvoyer une ou plusieurs lignes d'une seule colonne.

- Si la sous requête a renvoyé une seule valeur, vous pouvez lier les deux requêtes avec un opérateur de comparaison (=, >, <, ...).
- Si la sous requête a renvoyé plusieurs valeurs (ensemble de valeurs), vous pouvez lier les deux requêtes avec un opérateur de valeurs multiples (IN ou NOT IN).

▼ Sous-requête indépendante :

Dans les sous-requêtes indépendantes, l'exécution de la requête commence de la requête la plus interne vers la plus externe. L'exécution d'une requête interne est indépendante de la requête externe, mais le résultat de la requête interne est utilisé dans l'exécution de la requête externe.

Exemple 01 : Afficher le code et le nom du produit le plus cher.

```
SELECT code_P, nom_P FROM produit
WHERE prix = (Select Max(prix) From produit);
```

Dans ce cas, la sous requête sera exécutée en premier pour une seule fois (indépendamment de la requête principale), le résultat renvoyé par la sous requête (la valeur de prix la plus élevée) sera utilisé pour exécuter la requête principale.

▼ Sous-requête corrélée :

Une sous-requête corrélée est une sous-requête dont le résultat est différent selon les valeurs de la ligne de la requête externe pour laquelle la sous-requête est exécutée. Cela rend nécessaire d'exécuter la sous-requête pour chaque ligne extraite par la requête externe.

Exemple 02 : Afficher le produit avec le prix le plus cher dans chaque catégorie.

```
SELECT NomP, Prix, CodeCat
FROM Produit P1
WHERE Prix = (SELECT MAX(Prix) FROM Produit P2 WHERE P2.CodeCat=P1.CodeCat) ;
```

La requête principale doit exécuter la sous-requête pour chaque valeur distincte de P1 (et donc pour chaque ligne de la table Produit). Car chaque Produit peut appartenir à une catégorie différente.

▼ Sous-requête avec l'instruction : INSERT

On peut aussi utiliser les sous requêtes avec l'instruction INSERT. Dans l'instruction INSERT, les données renvoyées par la sous-requête sont utilisées pour être insérées dans une autre table.

Exemple 03 : Insérer le contenu de la table Produit dans une table Produit1.

```
INSERT INTO Produit1
        SELECT * FROM Produit;
```

Copier puis insérer le contenu de la table produit dans une table produit1.

▼ Sous-requête avec l'instruction : UPDATE || DELETE

Lorsqu'une sous-requête est utilisée avec l'instruction UPDATE, une ou plusieurs colonnes d'une table peuvent être mises à jour.

Exemple 04 : Appliquer une remise de 50% sur les produits dont le prix dépasse le prix moyen.

```
UPDATE Produit
SET prix = prix * 0,5
WHERE prix > ( SELECT AVG(prix) FROM Produit );
```

La clause : EXISTS

Cet opérateur permet de construire un prédicat évalué à vrai si la sous-requête renvoie au moins une ligne.

Exemple 01 : Afficher la liste des produits qui n'ont pas été commandé:

```
SELECT Code_P FROM Produit P
WHERE Not EXISTS (SELECT * FROM Commande C WHERE C.Code_P=P.Code_P);

Ou :

SELECT Code_P FROM Produit P
WHERE Not EXISTS (SELECT Code_P FROM Commande C WHERE C.Code_P=P.Code_P);
```

Ici, il s'agit de vérifier les produits qui n'existent pas dans la table Commande.

Une sous-requête **EXISTS** ignore les colonnes spécifiées par le **SELECT** de la sous-requête, car elles ne sont pas pertinentes.

Les deux requêtes ci-dessus produisent des résultats identiques.

PS. Si la sous requête est introduite par une clause **Exists/Not Exists**, il est possible qu'elle renvoie plusieurs colonnes (par exemple *select **) car dans ce cas, le résultat renvoyé par la sous requête ne sera pas utilisé pour faire une comparaison avec une valeur de l'autre côté (contrairement aux autres opérateurs: =, >, in, ...)

N.B : L'utilisation d'une jointure au lieu d'une sous-requête (lorsqu'il est possible) vous donne un gain de performances intéressant.

Le Tri du résultat d'un SELECT – La clause : ORDER BY



La clause Oracle Order By est utilisée pour trier les enregistrements renvoyés par une requête SELECT. Les deux ordres possibles sont: ASC (Ascendant) et DESC (Descendant).

Exemple :

Afficher les clients dans l'ordre alphabétique des noms (Ascendant).

```
SELECT Num_C1, Nom_C1
FROM Client
ORDER BY Nom_C1 ASC ;
```

Remarque: L'ordre par défaut est Ascendant.

Exemple 02 : Mettre un ordre suivant plusieurs colonnes

Donnez les noms et prix des produits par catégorie ; et pour chaque catégorie, par prix décroissant

```
SELECT Nom_P, Prix
FROM Produit
ORDER BY CodeCat ASC, Prix DESC ;
```



La clause GROUP BY est utilisée dans une instruction SELECT pour partitionner une table suivant un attribut qui vient après le « GROUP BY ». La clause GROUP BY renvoie une ligne par groupe.

Le GROUP BY est souvent utilisée avec des fonctions d'agrégation.

Exemple 01 :

Afficher le nombre de commandes passées par chaque client.

```
SELECT num_Cl, COUNT(*)  
FROM Commande  
GROUP BY num_Cl;
```

La table Commande sera partitionnée suivant les valeurs de l'attribut Num_Cl.

Exemple 02 :

Afficher le prix moyen des produits par catégorie.

```
SELECT Codecat, Avg(prix)  
FROM Produit  
GROUP BY Codecat;
```

La table Produit sera partitionnée suivant les valeurs de l'attribut Code_Cat.

➤ La clause : Having

La clause HAVING est utilisée avec la clause GROUP BY pour limiter les lignes renvoyées après le regroupement. Ça permet de ne renvoyer que les lignes qui remplissent la condition indiquée après la clause HAVING.

Exemple :

Afficher le prix moyen des produits par catégorie s'il dépasse 380.

```
SELECT Codecat, Avg(prix)  
FROM Produit  
GROUP BY Codecat  
HAVING Avg(prix) > 380;
```

► La forme générale d'une requête SELECT:

```
SELECT [DISTINCT] column_name(s)  
FROM table_name  
WHERE condition(s)  
GROUP BY column_name(s) HAVING condition(s)  
ORDER BY column_name(s) [ASC, DESC];
```

La clause : UNION

L'opération de l'union consiste à regrouper les résultats de plusieurs requêtes dans une seule requête.

Exemple :

Donner les noms des produits de la catégorie boisson ou bien ceux commandés par le client numéro 4.

```
SELECT NomP FROM Produit
WHERE CodeCat in (SELECT codeCat FROM Categorie WHERE nomCat='Boisson')

UNION

SELECT P.NomP FROM Produit P, Commande C
WHERE P.Code_P=C.Code_P AND C.numCl=4 ;
```

La première requête renvoie les produits de catégorie boisson. La deuxième requête renvoie les produits commandés par le client numéro 4. L'union combine les deux résultats.

Remarques :

- Les éléments dans les deux ensembles doivent être de même nature.
- L'opération de L'union élimine les doublons.

La clause : INTERSECT

Pour rechercher les éléments communs renvoyés par deux ou plusieurs requêtes.

Exemple :

Afficher les noms des produits commandés par Ahmed et Rym.

```
SELECT P.nom
FROM Produit P, Commande C, Client CL
WHERE P.Code_P=C.Code_P AND CL.Num_Cl=C.Num_Cl AND CL.nom='Ahmed'

INTERSECT

SELECT P.nom
FROM Produit P, Commande C, Client CL
WHERE P.Code_P=C.Code_P AND CL.Num_Cl=C.Num_Cl AND CL.nom='Rym';
```

Généralement, les requêtes avec la clause Insetsect sont remplacées par des requêtes avec la clause IN ou Exists. (C'est plus optimal)

Exercice :

Soit le schéma relationnel de la base de données «Etudiant » :

Etudiant (MatriculeEtu, nom_etu, prenom_etu, date_naissance, Adresse)

Unité (CodeUnité, libelle, nbr_heures, matricule_ens*)

Enseignant (MatriculeEns, nom_ens, prenom_ens)

Evaluation (MatriculeEtu*, CodeUnité *, note_CC, note_TP, note_examen)

1. Insérer les tuples ci-dessous : (utiliser les deux scripts ci-joints – Tables.sql & Donnees.sql).

Etudiant

Matricule_etu	nom_etu	prenom_etu	date_naissance	Adresse
202001	BOUSSAI	MOHAMED	12/01/2000	Alger
202002	CHAID	LAMIA	01/10/1999	Batna
202003	BRAHIMI	SOUAD	18/11/2000	Sétif
202004	LAMA	SAID	23/05/1999	Oran

Enseignant

Matricule_ens	nom_ens	prenom_ens
200001	HAROUNI	AMINE
200002	FATHI	OMAR
200003	BOUZIDANE	FARAH
200004	ARABI	ZOUBIDA

Evaluation

Matricule_etu	Code_Unite	note_CC	note_TP	note_examen
202001	U01	10	15	9
202002	U01	20	13	10
202004	U01	13	17	16
202002	U02	10	16	17
202003	U02	9	8	15
202004	U02	15	9	20
202002	U04	12	18	14
202003	U04	17	12	15
202004	U04	12	13	20

Unité

Code_Unité	libelle	nbr_heures	Matricule_ens
U01	WEB	6	200001
U02	BDD	6	200002
U03	RESEAU	3	200004
U04	SYSTEME	6	200003

2. Augmenter la note_CC de 2 pour tous les étudiants dont le nom commence par 'B'.
3. Remettre toutes les notes d'examen de l'unité "SYSTEME" à 0 pour tous les étudiants

❖ Exprimez en SQL les requêtes suivantes:

1. Quel est le nombre total d'étudiants ?
2. Comment s'appelle l'enseignant de module 'BDD' ?
3. Afficher les étudiants dans l'ordre alphabétique des noms.
4. Quel est le module avec le moins d'heures d'enseignement par semaine ?
5. Afficher les noms et prénoms des étudiants ayant obtenus des notes d'examens égales à 20.
6. Lister les modules (libellé) pour lesquels au moins un étudiant a obtenu une note de 15 à l'examen.
7. De deux manières différentes, affichez la moyenne des notes d'examen obtenues dans le module BDD.
(1^{ère} solution: avec AVG | 2^{ème} solution: en utilisant COUNT & SUM).
8. Affichez la moyenne des notes de TP obtenues au module enseigné par 'FATHI OMAR'.
9. Quels sont les modules dans lesquels aucun étudiant n'a obtenu une note d'examen supérieure à 10 ?
10. Afficher le nom et le prénom de l'étudiant ayant obtenu la meilleure note en examen 'BDD'.
11. Les modules sur lesquels tous les étudiants étaient examinés.
12. Les étudiants qui n'ont pas réussi à obtenir une note d'examen supérieure à 12 dans aucun des modules où ils ont été évalués ?
13. Pour chaque unité d'enseignement (libelle), afficher le nom et le prénom de l'étudiant ayant obtenu la meilleure note moyenne.
14. Les Les étudiants qui ont été examinés dans tous les modules sauf "RESEAU" et "SYS" ?